In practice, testing works by exercising the PDP directly. Each test constructs a representative authorization request—principal, action, resource, request context, and the relevant slice of entity data—and submits it to the PDP for evaluation. The test then asserts whether the resulting decision should be `allow` or `deny`. By testing policies as real authorization requests rather than as abstract logic, the team validates the entire decision process, including schema constraints, relationship traversal, global forbids, and interactions between multiple policies. This approach ensures that policy changes are verified in the same way they will be enforced in production, reducing surprises when new rules are deployed.

Approaches to testing include the following:

- Unit tests check individual rules in isolation. An example is ensuring that an owner can always edit their own document, or that a viewer cannot share it.
- Scenario tests run end-to-end cases involving multiple policies and relationships. ACME performs scenarios such as “Jack (a customer) shares the Q3 plan with Bob (his account manager)” to verify that delegation works correctly.
- Regression tests document past bugs to prevent them from happening again. After the “manager of an owner” bug, ACME added a test to ensure managers always inherit the correct edit permissions.
- Coverage checks ensure that every major rule has at least one related test, so no part of the policy set goes unchecked.

Because Cedar’s evaluation engine is embeddable, these tests run quickly within ACME’s CI pipelines. Policies are just code, but they have a b*ig impact*: a single wrong condition can accidentally grant access to everyone or block an entire team.

Writing tests for authorization policies follows its own best practices. Unlike application logic, policies are concise but important: One incorrect condition can accidentally grant access or block legitimate work.

These are some strategies for running effective tests: